



Documentación Técnica

Zocorn Movies

Zocorn Movies

1. Descripción general

Zocorn es una aplicación web de tipo SPA (Single Page Application) para explorar películas y series de TV, construida con Vue 3 y Vuetify, que consume la API pública de TMDB a través de funciones serverless propias que actúan como proxy (ocultando el token de TMDB al cliente).

No dispone de backend propio de usuarios: no hay login, registro ni base de datos. Todo el estado de usuario (favoritos, filtros, avisos leídos) se persiste en `localStorage` del navegador.

2. Stack tecnológico

Categoría	Tecnología	Versión
Framework	Vue 3 (Composition API, <code><script setup></code>)	^3.5.25
Lenguaje	TypeScript	~5.9.3
Build tool	Vite	^8.0.0-beta.13
UI Framework	Vuetify 3	^3.11.8
Estilos utilitarios	Tailwind CSS	^4.2.0
Routing	Vue Router 4 (modo <code>history</code>)	^4.6.4
Tipografías	@fontsource/montserrat, @fontsource/roboto	—
Iconos	Material Design Icons (@mdi/font)	^7.4.47
Scroll suave	Lenis	^1.3.17
Backend	Funciones serverless (@vercel/node) — proxy a TMDB	^5.10.1
Deploy	Vercel	—

3. Arquitectura

```

api/                                Funciones serverless (proxy a TMDB)
% % % _lib/tmdb.ts                  Helper común: fetch a TMDB con bearer token
% % % movies/                      Endpoints de películas (popular, top_rated, search,
%                                discover, genres, [movieId]/*, trending/[timeWindow])
% % % tv/                          Mismos endpoints, para series
% % % people/trending/[timeWindow].ts

src/
% % % assets/                      Logos, imágenes y recursos estáticos
% % % components/                  Componentes reutilizables (PrimeNavbar,
%                                MoviesScroller, LoadingResult, VueLadda...)
% % % composables/                Lógica reutilizable con estado compartido
% % % useBookmarkedMovies.ts
% % % useBookmarkedSeries.ts
% % % useToast.ts
% % % useYoutubeLoopingBackground.ts
% % % models/                     Tipos/interfaces TypeScript del dominio
% % % router/                     Configuración de rutas (Vue Router)
% % % services/tmdbApiService.ts  Cliente HTTP hacia /api/movies y /api/tv
% % % views/                      Vistas de la app (una carpeta por vista)

```

La arquitectura sigue el patrón **cliente ligero + BFF serverless**: el frontend nunca llama directamente a la API de TMDB; siempre pasa por las funciones en **api/**, que añaden el **TMDB_BEARER_TOKEN** (variable de entorno privada) y reenvían la petición.

Cada vista (**view**) está encapsulada en su propia carpeta con su plantilla y su hoja de estilos:

```

src/views/NombreView/
% % % NombreView.vue    # Template + lógica
% % % NombreView.css    # Estilos propios

```

DetailedSeries y **SearchSeries** reutilizan el CSS de **DetailedMovie** y **Search** respectivamente, ya que comparten la misma maquetación.

4. Rutas de la aplicación

Ruta	Nombre	Vista	Descripción	
/	home	Home.vue	Portada: hero, tendencias, populares, mejor valoradas, próximos estrenos	
/buscarmovies	search	Search.vue	Buscador de películas con filtros avanzados	
/buscarseries	search-series	SearchSeries.vue	Buscador de series con filtros avanzados	
/movie/:idtmdb	detailed-movie	DetailedMovie.vue	Ficha de detalle de una película	
/tv_show/:idtmdb	detailed-series	DetailedSeries.vue	Ficha de detalle de una serie	
/mis-peliculas	my-movies	MyMovies.vue	Listado de películas guardadas	
/mis-series	my-series	MySeries.vue	Listado de series guardadas	
/politica-privacidad-condiciones-uso	privacy-policy	PrivacyPolicy.vue	Aviso legal / política de privacidad	

El enrutador usa `createWebHistory` y resetea el scroll al top en cada navegación.

5. Capa de datos (API)

5.1 Backend serverless (api/)

- `api/_lib/tmdb.ts`: helper común que realiza el `fetch` contra la API de TMDB inyectando el `Authorization: Bearer <TMDB_BEARER_TOKEN>`.
- `api/movies/`: endpoints de películas — populares, mejor valoradas, búsqueda (`search`), descubrimiento con filtros (`discover`), géneros, y sub-recursos por `[movieId]` (detalle, créditos, vídeos, imágenes, recomendadas, similares, reviews). Incluye también `trending/[timeWindow]`.
- `api/tv/`: los mismos endpoints, adaptados al recurso `tv` de TMDB.
- `api/people/trending/[timeWindow].ts`: personas en tendencia.

5.2 Cliente HTTP (src/services/tmdbApiService.ts)

Encapsula las llamadas `fetch` desde el frontend hacia las rutas `/api/movies/*` y `/api/tv/*`, sin exponer nunca el token de TMDB al navegador.

6. Gestión de estado (composables)

El estado compartido se implementa con `ref()` a nivel de módulo (patrón singleton de Composition API), sin librería de estado externa (no Pinia/Vuex).

6.1 useBookmarkedMovies / useBookmarkedSeries

- Persisten en `localStorage` bajo las claves `bookmarked-movies` / `bookmarked-series`.
- Límite máximo: **20 elementos** por tipo. Al superarlo se muestra un toast de aviso.
- API expuesta: `bookmarkedMovies` (ref), `isBookmarked(id)`, `addBookmark(item)`, `removeBookmark(id)`, `toggleBookmark(item)`, `clearBookmarks()`.

6.2 useToast

Snackbar/notificación global reutilizada en toda la app para mensajes de éxito, error o aviso.

6.3 useYoutubeLoopingBackground

Gestiona el trailer de YouTube en bucle usado como fondo del header/hero.

7. Variables de entorno

Variable	Descripción	Dónde se usa
TMDB_BEARER_TOKEN	Token de lectura (v4) de la API de TMDB	Solo en <code>api/</code> (funciones serverless), nunca en el cliente

Se configura copiando `.env.example` a `.env.local` en desarrollo, y como variable de entorno en el dashboard de Vercel en producción.

8. Scripts disponibles

Comando	Descripción
npm install	Instala dependencias
npm run dev	Arranca Vite en modo desarrollo (http://localhost:5173)
npm run build	Compila TypeScript (<code>vue-tsc -b</code>) y genera el build de producción
npm run build:pages	Build de producción con <code>--base /</code>
npm run preview	Sirve el build de producción localmente
npm run new:view NombreView	Genera <code>src/views/NombreView/NombreView.vue</code> y <code>.css</code> mediante <code>scripts/create-view.mjs</code>

Nota: para probar también las funciones de `api/` en local es necesario usar `vercel dev`, que levanta el proxy de TMDB junto con Vite.

9. Despliegue

Proyecto pensado para desplegarse en **Vercel**:

- Detecta automáticamente el framework Vite (build) y aplica el fallback SPA para las rutas de `vue-router`.
- Detecta y despliega las funciones serverless de `api/`.
- Único requisito: configurar `TMDB_BEARER_TOKEN` en las variables de entorno del proyecto en Vercel.
- Configuración adicional de rutas/build en `vercel.json`.

10. Convenciones de código

- Composition API con `<script setup lang="ts">` en todos los componentes y vistas.
- Cada vista vive en su propia carpeta con plantilla y CSS separados (`NombreView.vue` + `NombreView.css`).
- Los tipos de dominio (películas, series, etc.) se centralizan en `src/models/`.
- El estado compartido se implementa como composables con `ref()` a nivel de módulo, no como componentes con props/emits para datos globales (favoritos, toast).
- Nuevas vistas se generan con `npm run new:view` y se registran manualmente en `src/router/index.ts`.

11. Limitaciones conocidas

- No existe autenticación ni backend propio de usuarios: los favoritos son locales al navegador/dispositivo y se pierden al borrar `localStorage`.

- Sin sincronización entre dispositivos.
- Límite fijo de 20 favoritos por tipo (película/serie), sin posibilidad de configurarlo desde la interfaz.